Step 2d — Varianti della domanda: un prompt fisso, poche varianti opzionali

Obiettivo

Decidere quanta variazione di formulazione serve quando la domanda è sempre la stessa. Nel piano generale (../fine_tuning/02d_varianti_domanda.md) si generavano ~40–60 varianti per domanda e nomi di campo alternativi, perché chi integra un decision model scrive le domande a modo suo. Qui il chiamante è il nostro codice: **la domanda è fissa**.

1. Default: una domanda, identica ovunque

La domanda si sceglie sul nostro dev (../02d_varianti_domanda.md) fra quelle dello studio focalizzato, già nei file di `config/questions/`, e si usa identica in training, valutazione e produzione:

ID	Testo (senza la frase finale comune)	P(supportato)
Q1 (`support_simple`)	"Does 'document' support 'claim'?"	P(sì)
Q2 (`unsupported_detail`)	"Does 'claim' contain any factual detail that 'document' does not establish? …", con le descrizioni delle opzioni	1 − P(sì)
Q4 (`support_every_part`)	"Is every part of 'claim' stated in or directly implied by 'document'?"	P(sì)

Regola: AUC macro più alta in media sui 10 modelli; a parità, soglia più vicina a 0,5; a parità, la più corta. Una domanda per tutti i modelli; ognuno la vede nel proprio formato.

Perché fissare la domanda: con un solo prompt l'adapter LoRA può concentrare il cambiamento dove serve. La fragilità ("funziona solo con questa stringa") non è un problema se la stringa non cambia mai.

2. Quando servono comunque le varianti

Situazione	Cosa fare
Il modello sarà chiamato da altri con domande proprie	varianti al 20–30% (come il piano generale)
L'input arriva con nomi di campo diversi (`context`/`answer`, `source`/`statement`)	normalizzare i nomi **prima** del modello, nel codice che lo chiama: è più semplice e sicuro che insegnarli
Si vuole un effetto di regolarizzazione	10% di varianti (esperimento P-VAR): il segnale "supportato o no" non deve dipendere dalle parole esatte

3. Se si usano: come generarle

Stessa procedura del piano generale, in piccolo:

1. un LLM scrive ~20 riformulazioni di Q (stesso significato: "no" anche quando manca l'evidenza);
2. filtro con il modello di partenza (KnowLine-4B): si scartano le varianti la cui AUC su 300 righe di `dev_select` è più di 2 punti sotto Q;
3. ~5 varianti tenute da parte per misurare la stabilità (06).

4. Field names e formato dello stato

Restano quelli nativi (`document`, `claim`) e il formato di ciascun modello, prodotto dal codice dei suoi autori. Cambiarli non porta nulla e allontana il prompt da quello che il modello conosce.

Criteri per passare allo step successivo

- Domanda di produzione scelta e scritta in `finetune/question.json` (testo, descrizioni, ordine delle opzioni per modello).
- Deciso se si fa l’esperimento P-VAR; in caso, varianti generate e filtrate.